Presentation Guide

Project Category: B1AI DEVOP1
Author: Roni Fernandes Dias / François LANGE
Date: June 25, 2026

Table of Contents

1. Project Timeline & Progress 3

2. Hosting Environment 3

3. Source Code Repository 3

4. Infrastructure Overview 4

5. Containerized Components & Demonstration Guide 4

A. Web Application (Frontend/Backend) 4

B. MySQL Database 5

C. Zabbix (Monitoring & Visualization) 5

D. Apache Airflow (Workflow Automation - Optional/Bonus) 5

E. Jenkins (CI/CD Automation) 5

F. Kubernetes (Orchestration Layer) 6

Project Antigravity (DEVOP1) - Presentation Guide

This document serves as a cheat sheet and guide for your project presentation. It outlines where to find resources, how to access them, useful commands, and where logs are located for each component.

---

1. Project Timeline & Progress

Objective: Present development progress, major milestones, and phases.

- **Where to find it:**
- **Taiga (Agile Board):** The primary source of truth for tasks, user stories, and sprints.
- **Sprint Definitions:** Located locally in `taiga-sprints.yml`.
- **Documentation:** Core project documents like `Retro_Planning.md`, `User stories.md`, and `Wireframes.md` are in the project root.
- **Daily Progress Logs:** Synced to the `documentation/` directory and Taiga Wiki automatically via `ci-cd/antigravity_bot.py`.

---

2. Hosting Environment

Objective: Demonstrate where your project is currently hosted.

- **Local Development (Docker Compose):**
- The entire local stack is orchestrated via `docker-compose.yml`.
- **Start the environment:** `docker compose up -d`
- **Check status:** `docker compose ps`
- **Environment Configuration:** All ports and credentials are set in the `.env` file.
- **Production/Staging (Kubernetes - K3s):**
- The application is deployed to a K3s Kubernetes cluster for higher environments.
- Configuration manifests are stored in `kubernetes/manifests/`.

---

3. Source Code Repository

Objective: Share and walk through the Git repository, branches, and CI/CD.

- **Repository Structure:**
- `app/`: Contains frontend, backend, Airflow DAGs, and logs.

- `kubernetes/`: K8s manifests.
- `ci-cd/`: Scripts for Taiga/Git integration.
- `database/scripts/`: MySQL initialization scripts.
- **Branching Strategy & Flow:**
- Code flows through `development` `test` `production`.
- Feature branches must match Taiga task IDs: `feature/TG--description`.
- **CI/CD Integration:**
- The **Jenkins** pipeline is defined in the `Jenkinsfile` at the project root. It uses dynamic Kubernetes agents to build Docker images and deploy via `kubectl`.
- **Useful Git Commands:**
- Create a feature branch: `git checkout -b feature/TG--description`
- Commit with Taiga ID: `git commit -m "TG-: "`

---

4. Infrastructure Overview

Objective: High-level overview and diagrams.

- **Architecture Diagram:** Open `README.md` to show the comprehensive Mermaid diagram detailing the network, Docker Compose, and Kubernetes topology.
- **Data Flow Summary:** Developers and Users hit the Load Balancer/Web App, which communicates with the consolidated MySQL database. Zabbix monitors the stack, while Airflow automates workflows.
- **Infrastructure as Code:**
- Docker Compose (`docker-compose.yml`)
- Kubernetes Manifests (`kubernetes/`)
- Jenkins Pipeline (`Jenkinsfile`)

---

5. Containerized Components & Demonstration Guide

Objective: Identify, explain, and demo each service.

A. Web Application (Frontend/Backend)

- **Role:** The core Flask/React application processing requests and serving the UI.
- **How to Access (Local):** `http://localhost:` (Default: `5000`).
- **Code Location:** `./app/backend/` and `./app/frontend/`.
- **Logs:**

- File: Mapped locally to `./app/logs/`.
- Docker: `docker logs devop1-backend`

B. MySQL Database

- **Role:** Consolidated database hosting `devop1_db` (App), `airflow_db` (Workflows), and `zabbix_db` (Monitoring).
- **How to Access (Local):**
- **Host:** `127.0.0.1` | **Port:** `3317` (Mapped from `3306`).
- **Credentials:** User: `dev_user`, Password from `.env`.
- **Useful Command:** `mysql -h 127.0.0.1 -P 3317 -u dev_user -p`
- **Logs:** `docker logs devop1-mysql`

C. Zabbix (Monitoring & Visualization)

- **Role:** Telemetry, metric gathering, and alert visualization.
- **How to Access:** `http://localhost:` (Check `.env` for port mapping).
- **Logs:**
- Server: `docker logs devop1-zabbix-server`
- Web UI: `docker logs devop1-zabbix-web`

D. Apache Airflow (Workflow Automation - Optional/Bonus)

- **Role:** Orchestrates automated background jobs (DAGs).
- **How to Access:** `http://localhost:`
- **Credentials:** Configured in `.env` (default usually admin/admin).
- **Logs:** Mapped to `./app/logs/` or via `docker logs airflow-webserver`.

E. Jenkins (CI/CD Automation)

- **Role:** Automates build and Kubernetes deployments. Runs as a Master Pod in K3s.
- **How to Access:** Via your Jenkins Web UI URL.
- **Where to find Logs:**
- Pipeline logs are available directly in the Jenkins UI under the specific **Build Console Output**.
- The `Jenkinsfile` controls this logic.

F. Kubernetes (Orchestration Layer)

- **Role:** Hosts the CI/CD pipeline agents and production web applications.
- **Useful Commands for Demo (`kubectl`):**
- **List all Pods:** `kubectl get pods -A`
- **List Services (Networking):** `kubectl get svc`
- **List Deployments:** `kubectl get deployments`
- **View Pod Logs:** `kubectl logs `
- **Describe a Pod (Troubleshooting):** `kubectl describe pod `
- **Apply Manifests:** `kubectl apply -f kubernetes/manifests/`

Page Search Index

Airflow ..... Page(s): 3, 4, 5

Docker ..... Page(s): 3, 4, 5

Jenkins ..... Page(s): 4, 5

Kubernetes ..... Page(s): 3, 4, 5, 6

MySQL ..... Page(s): 4, 5

Telemetry ..... Page(s): 5

Zabbix ..... Page(s): 4, 5